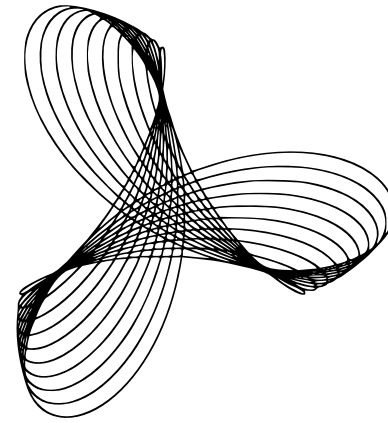
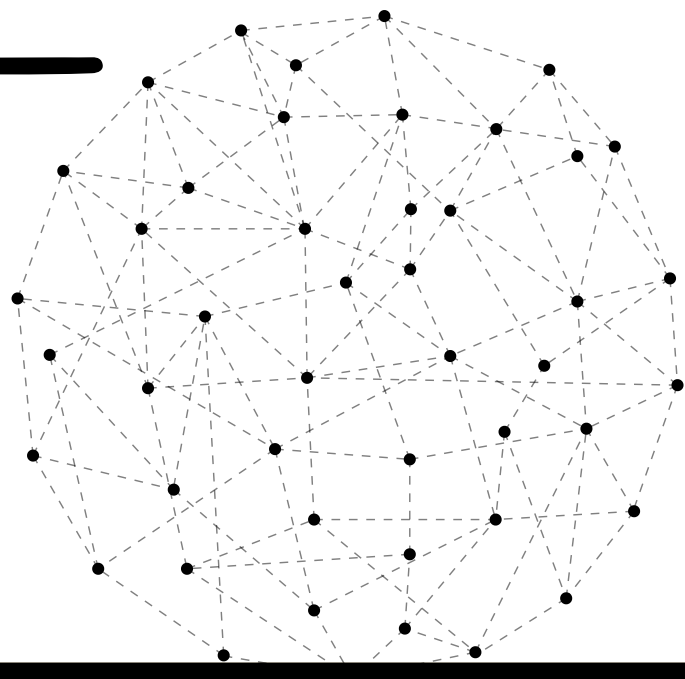




XAI for Transformers: Even Better Explanations through Interpolated Propagation

By: - Nikhil Goyal



[https://github.com/nikhil-405/XAI for Transformers](https://github.com/nikhil-405/XAI_for_Transformers)

CHALLENGE

GI: captures sensitivity, breaks completeness

LRP: satisfies completeness, lose expressiveness

ie, a binary decision

PROPOSED SOLUTION

Interpolated Propagation

PROPOSED SOLUTION

Interpolated Propagation

Allow partial gradient flow:
parameter γ

PROPOSED SOLUTION

Interpolated Propagation

$\gamma = 0 \rightarrow$ no gradient flow (LRP) [Ali et al.]

$\gamma = 1 \rightarrow$ complete Gradient flow (GI)

$$0 < \gamma < 1$$

THEORETICAL BOUNDS

THEORETICAL BOUNDS

ATTENTION HEADS

THEORETICAL BOUNDS

ATTENTION HEADS

We do,



Interpolated Gradient flow for AH

```
attention_probs = (1 - gamma) * attn.detach() + gamma * attn
```


THEORETICAL BOUNDS

ATTENTION HEADS

Therefore,

$$\tilde{p}_{ij} = (1-\gamma) p_{ij}^{[\text{detach}]} + \gamma p_{ij}$$

$$\frac{\partial \tilde{p}_{kj}}{\partial x_i} = \gamma \frac{\partial p_{kj}}{\partial x_i}$$

{ completeness breach = $\gamma \cdot (\text{original breach})$ }

THEORETICAL BOUNDS

ATTENTION HEADS

$$y_j = \sum_i x_i p_{ij}.$$

GI relevance

$$R(x_i) = x_i^T \frac{\partial f}{\partial x_i}$$
$$R(y_j) = y_j^T \frac{\partial f}{\partial y_j}$$

In order to compute relevance, we need the gradients,

THEORETICAL BOUNDS

ATTENTION HEADS

Consider softmax,

$$p_{ij} = \frac{\exp(q_{ij})}{\alpha_j} \quad \alpha_j = \sum_i \exp(q_{ij})$$

THEORETICAL BOUNDS

ATTENTION HEADS

Consider softmax,

$$p_{ij} = \frac{\exp(q_{ij})}{\alpha_j} \quad \alpha_j = \sum_i \exp(q_{ij})$$

Differentiate with respect to the i th token, x_i , you get

THEORETICAL BOUNDS

ATTENTION HEADS

$$\begin{aligned}\frac{\partial p_{kj}}{\partial x_i} &= \frac{1}{\alpha_j^2} \left(\exp(q_{kj}) \frac{\partial q_{kj}}{\partial x_i} \alpha_j - \exp(q_{kj}) \exp(q_{ij}) \frac{\partial q_{ij}}{\partial x_i} \right) \\ &= p_{kj} \frac{\partial q_{kj}}{\partial x_i} - p_{kj} p_{ij} \frac{\partial q_{ij}}{\partial x_i} \\ &= p_{kj} [\delta_{ik} - p_{ij}] \frac{\partial q_{ij}}{\partial x_i}\end{aligned}$$

THEORETICAL BOUNDS

ATTENTION HEADS

Similarly, differentiating wrt target token x'_j gives us:

$$\begin{aligned}\frac{\partial p_{il}}{\partial x'_j} &= \frac{1}{\alpha_l^2} \left(\exp(q_{il}) \frac{\partial q_{il}}{\partial x'_j} \alpha_l \right. \\ &\quad \left. - \exp(q_{il}) \delta_{jl} \sum_k \exp(q_{kl}) \frac{\partial q_{kl}}{\partial x'_j} \right) \\ &= p_{il} \frac{\partial q_{il}}{\partial x'_j} - p_{ij} \delta_{jl} \sum_k p_{kj} \frac{\partial q_{kj}}{\partial x'_j} \\ &= \delta_{jl} p_{ij} \left(\frac{\partial q_{ij}}{\partial x'_j} - \mathbb{E}_j \left[\frac{\partial q_{:j}}{\partial x'_j} \right] \right)\end{aligned}$$

THEORETICAL BOUNDS

ATTENTION HEADS

$$y_j = \sum_i x_i p_{ij}.$$

Using the multivariate chain rule, we get:

$$\begin{aligned} \frac{\partial f}{\partial x_i} &= \sum_j \left(\frac{\partial y_j}{\partial x_i} + \sum_k \frac{\partial p_{kj}}{\partial x_i} \frac{\partial y_j}{\partial p_{kj}} \right) \frac{\partial f}{\partial y_j} \\ &= \sum_j p_{ij} \frac{\partial f}{\partial y_j} + \sum_{jk} p_{kj} [\delta_{ik} - p_{ij}] \frac{\partial q_{ij}}{\partial x_i} x_k^\top \frac{\partial f}{\partial y_j} \\ &= \sum_j p_{ij} \frac{\partial f}{\partial y_j} + \sum_j p_{ij} \frac{\partial q_{ij}}{\partial x_i} (x_i - \mathbb{E}_j[x])^\top \frac{\partial f}{\partial y_j} \end{aligned}$$

THEORETICAL BOUNDS

ATTENTION HEADS

We repeat this for the target embedding, and get

$$\begin{aligned}\frac{\partial f}{\partial x'_j} &= \sum_l \sum_i \frac{\partial p_{il}}{\partial x'_j} \frac{\partial y_l}{\partial p_{il}} \frac{\partial f}{\partial y_l} \\ &= \sum_l \sum_i \delta_{jl} p_{ij} \left(\frac{\partial q_{ij}}{\partial x'_j} - \mathbb{E}_j \left[\frac{\partial q_{:j}}{\partial x'_j} \right] \right) x_i^\top \frac{\partial f}{\partial y_l} \\ &= \sum_i p_{ij} \left(\frac{\partial q_{ij}}{\partial x'_j} - \mathbb{E}_j \left[\frac{\partial q_{:j}}{\partial x'_j} \right] \right) x_i^\top \frac{\partial f}{\partial y_j} \\ &= \mathbb{E}_j \left[\left(\frac{\partial q_{:j}}{\partial x'_j} - \mathbb{E}_j \left[\frac{\partial q_{:j}}{\partial x'_j} \right] \right) x^\top \right] \frac{\partial f}{\partial y_j}.\end{aligned}$$

THEORETICAL BOUNDS

ATTENTION HEADS

Use the GI relevance formulae, ie,

GI relevance

$$R(x_i) = x_i^T \frac{\partial f}{\partial x_i}$$
$$R(y_j) = y_j^T \frac{\partial f}{\partial y_j}$$

THEORETICAL BOUNDS

ATTENTION HEADS

Summing the two scores, we get:

$$\begin{aligned} \sum_i \mathcal{R}(x_i) + \sum_j \mathcal{R}(x'_j) \\ = \sum_j \mathcal{R}(y_j) + \sum_j \left(\mathbb{E}_j \left[(q_{:j} - \mathbb{E}_j[q_{:j}]) x^\top \right] \right. \\ \left. + \mathbb{E}_j \left[q_{:j} \cdot (x - \mathbb{E}_j[x])^\top \right] \right) \frac{\partial f}{\partial y_j} \end{aligned}$$

THEORETICAL BOUNDS

ATTENTION HEADS

assume zero centered distribution, ie, zero expected values, and we end up with:

$$\begin{aligned} \sum_i \mathcal{R}(x_i) + \sum_j \mathcal{R}(x'_j) \\ = \sum_j \mathcal{R}(y_j) + \sum_j 2 \text{Cov}_j(q_{:j}, x)^\top \frac{\partial f}{\partial y_j} \end{aligned}$$

THEORETICAL BOUNDS

ATTENTION HEADS

Therefore, for the interpolated case, we have:

$$\sum_i R(x_i) + \sum_j R(x'_j) = \sum_j R(y_j) + 2\gamma \sum_j \text{Cov}_j(v_{:,j}, x)^T \frac{\partial f}{\partial y_j}$$

THEORETICAL BOUNDS

Layer Norm

THEORETICAL BOUNDS

Layer Norm

The core part of LayerNorm can be decomposed into two parts:

$$\tilde{x}_i = x_i - \mathbb{E}[x] \quad (10)$$

$$y_i = \frac{\tilde{x}_i}{\sqrt{\epsilon + \mathbb{E}[\tilde{x}^2]}} \quad (11)$$

THEORETICAL BOUNDS

Layer Norm

In code, we have:

```
Interpolated Gradient flow for LN
1 mean = (1 - gamma)*mean.detach() + gamma*mean
2 std = (1 - gamma)*std.detach() + gamma*std
```

THEORETICAL BOUNDS

Layer Norm

For the centering step,

$$\sum_i \mathcal{R}(x_i) = \sum_i \sum_j \frac{\partial \tilde{x}_j}{\partial x_i} \frac{x_i}{\tilde{x}_j} \mathcal{R}(\tilde{x}_j) \quad (12)$$

Observing that $\tilde{x}_j(x)$ is a homogeneous function of degree one, and that consequently $\sum_i x_i \frac{\partial \tilde{x}_j}{\partial x_i} = \tilde{x}_j$ for all j , Eq. (12) reduces to

$$\sum_i \mathcal{R}(x_i) = \sum_i \mathcal{R}(\tilde{x}_i) \quad (13)$$

ie, conservation satisfied for the centering step,

THEORETICAL BOUNDS

Layer Norm

For the rescaling step,

$$\begin{aligned}\frac{\partial y_j}{\partial \tilde{x}_i} &= \frac{\delta_{ij}}{\sqrt{\epsilon + \mathbb{E}[\tilde{x}^2]}} - \frac{\tilde{x}_j \frac{\partial}{\partial \tilde{x}_i} (\sqrt{\epsilon + \mathbb{E}[\tilde{x}^2]})}{(\sqrt{\epsilon + \mathbb{E}[\tilde{x}^2]})^2} \\ &= \frac{\delta_{ij}}{\sqrt{\epsilon + \mathbb{E}[\tilde{x}^2]}} - \frac{\frac{1}{N} \tilde{x}_i \tilde{x}_j}{(\sqrt{\epsilon + \mathbb{E}[\tilde{x}^2]})^3}\end{aligned}$$

THEORETICAL BOUNDS

Layer Norm

$$\begin{aligned}\sum_i \mathcal{R}(\tilde{x}_i) &= \sum_i \sum_j \frac{\partial y_j}{\partial \tilde{x}_i} \frac{\tilde{x}_i}{y_j} \mathcal{R}(y_j) \\ &= \sum_i \sum_j \left(\frac{\delta_{ij}}{\sqrt{\epsilon + \mathbb{E}[\tilde{x}^2]}} - \frac{\frac{1}{N} \tilde{x}_i \tilde{x}_j}{(\sqrt{\epsilon + \mathbb{E}[\tilde{x}^2]})^3} \right) \frac{\tilde{x}_i}{y_j} \mathcal{R}(y_j) \\ &= \sum_i \sum_j \left(\delta_{ij} y_i - \frac{1}{N} y_i y_i y_j \right) \cdot \frac{1}{y_j} \mathcal{R}(y_j) \\ &= \sum_j (1 - \mathbb{E}[y^2]) \mathcal{R}(y_j) \\ &= \left(1 - \frac{\mathbb{E}[\tilde{x}^2]}{\epsilon + \mathbb{E}[\tilde{x}^2]} \right) \sum_j \mathcal{R}(y_j)\end{aligned}$$

THEORETICAL BOUNDS

Layer Norm

Since LRP is satisfied by the centering step, the overall contribution is due to the re-scaling contribution:

$$\sum_i \mathcal{R}(x_i) = \left(1 - \frac{\text{Var}[x]}{\epsilon + \text{Var}[x]}\right) \sum_i \mathcal{R}(y_i)$$

THEORETICAL BOUNDS

Layer Norm

Therefore, for the interpolated gradient flow setting:

$$\sum_i R(x_i) = \left(1 - \frac{\text{Var}[x]}{\varepsilon + \text{Var}[x]} \right) \gamma \sum_i R(y_i)$$

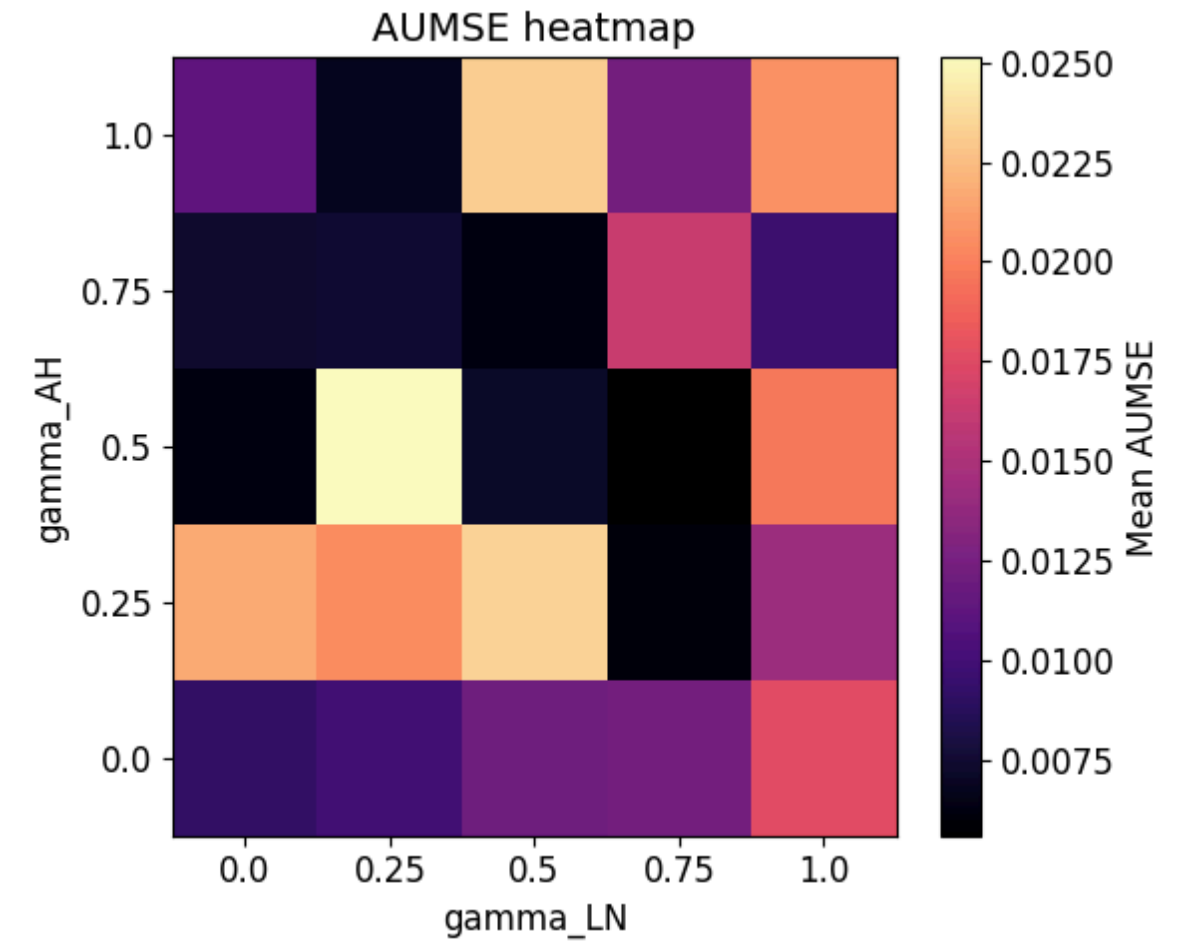
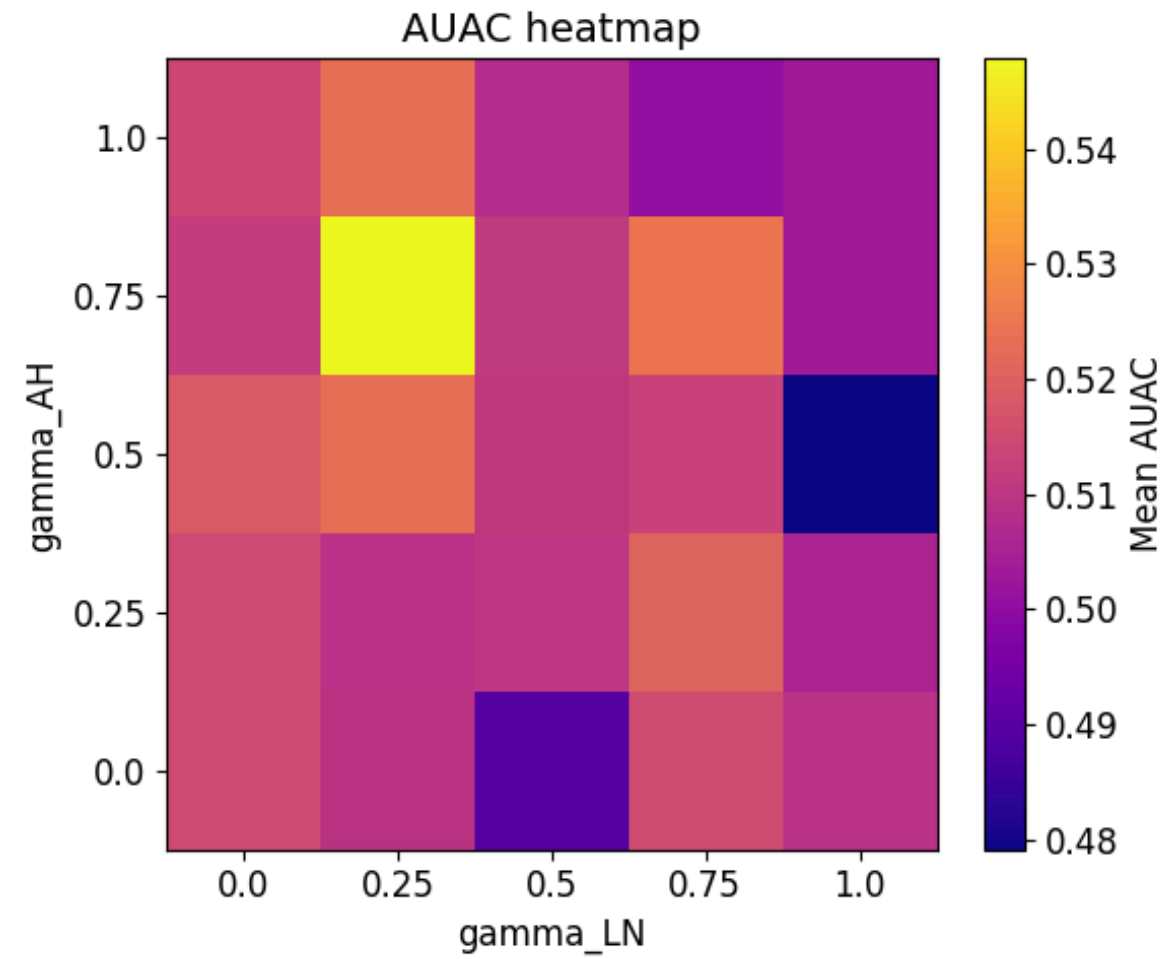
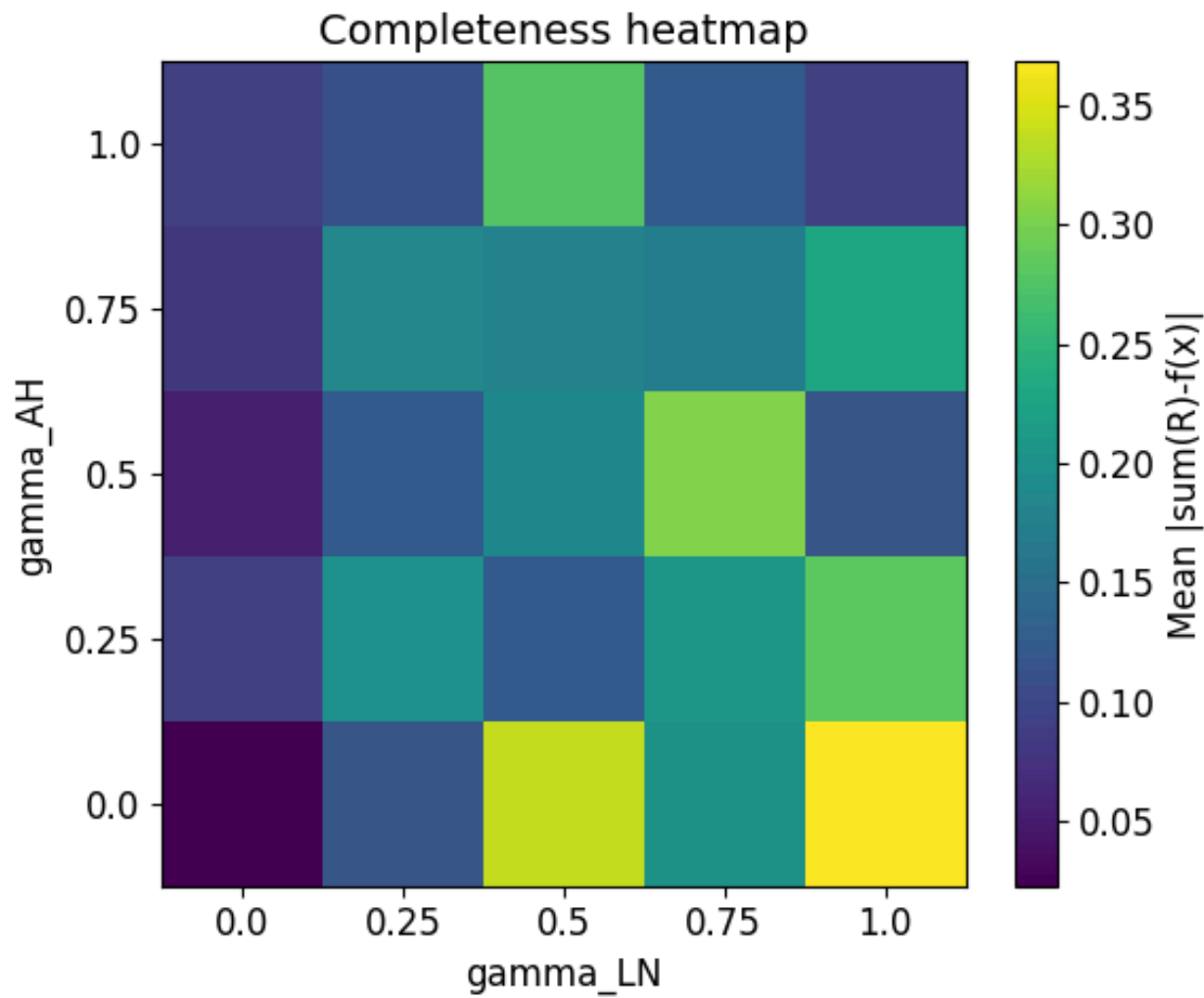
Solution 1

GLOBAL GATING

First, for all the AHs, and LNs, we use the parameters, γ_{AH} , and γ_{LN} . We grid search over $[0, 0.25, 0.5, 0.75, 1]$

ie, a total of 25 possible combinations

Observations



Observations

AUAC — Area Under the Activation Curve

A metric that measures how quickly adding the most relevant features (tokens/nodes) increases the model's confidence in the correct class. **Higher AUAC = more faithful explanations.**

AUMSE — Area Under the Mean-Squared Error Curve

A metric that measures how little the prediction changes when the least relevant features are removed. **Lower AUMSE = more robust and precise explanations.**

Observations

3 Pareto optimal combinations:

Observations

3 Pareto optimal combinations:

- $(0, 0) \rightarrow$ best completeness
- $(0.25, 0.75) \rightarrow$ best AUMSE
- $(0.75, 0.25) \rightarrow$ best AUAC

Observations

What is more dominant AH or LN?

Observations

What is more dominant AH or LN?

	Metric	Pearson vs γ_{AH}	Pearson vs γ_{LN}	Spearman vs γ_{AH}	Spearman vs γ_{LN}	Dominant Factor
0	Completeness	-0.244044	0.569342	-0.223572	0.631494	gamma_LN
1	AUAC	0.139916	-0.430984	0.043146	-0.505980	gamma_LN
2	AUMSE	-0.056083	0.158054	-0.078446	0.164738	gamma_LN

For all 3 evaluation metrics, γ_{LN} is “causing more problems”, ie,

“Layer Norm is the primary bottleneck in explanations.”

Observations

There yet exists, other sources of LRP errors, primarily due to the fact that transformers:

- Still has residual connections
- Still has feedforward layers
- Still has classifier biases
- Still has untuned pooling
- Still uses frozen embeddings

Solution 2

- Instead of grid search, we learn these parameters
- Instead of global gating, we use layer-level gating (allowing for more fine-tuned control)

GammaNet

- Simple MLP
- Learns layer-wise parameters
- Input: [CLS] embeddings
- Output: [1, 2, L]
- Loss function:

GammaNet

- Simple MLP
- Learns layer-wise parameters
- Input: [CLS] embeddings
- Output: [1, 2, L]
- Loss function:

$$\text{Loss}(L) = \lambda_c \underbrace{\left(f(x) - \mathbb{E} R \right)^2}_{\substack{\downarrow \\ \text{completeness} \\ \text{error}}} - \lambda_a (\text{AVAC}_{\text{proxy}}) + \| \gamma \|$$

GammaNet

$$\text{Loss}(L) = \lambda_c \underbrace{(f(x) - \varepsilon R)^2}_{\substack{\downarrow \\ \text{completeness} \\ \text{error}}} - \lambda_a (\text{AVAC proxy}) + \| \gamma \|$$

GammaNet

$$Loss(L) = \lambda_c \underbrace{(f(x) - \varepsilon R)^2}_{\substack{\downarrow \\ \text{completeness} \\ \text{error}}} - \lambda_a (AVAC_{\text{proxy}}) + ||\gamma||$$

In theory,

$$\max_{\theta} AVAC(\theta) \text{ s.t. } comp(\theta) \leq \varepsilon$$

$$L(\theta, \lambda) = -AVAC(\theta) + \lambda (comp(\theta) - \varepsilon)$$

[Lagrangian]

GammaNet

$$Loss(L) = \lambda_c \underbrace{(f(x) - \varepsilon R)^2}_{\substack{\downarrow \\ \text{completeness} \\ \text{error}}} - \lambda_a (AVAC_{\text{proxy}}) + ||\gamma||$$

In practice,

$$\frac{\partial L}{\partial \gamma} = \lambda_c \frac{\partial}{\partial \gamma} (f(x) - \varepsilon R)^2 + \frac{\partial}{\partial \gamma} (||\gamma||) - \lambda_a \underbrace{\frac{\partial}{\partial \gamma} (AVAC)}_{\substack{\downarrow \\ = 0 \\ (.no_grad())}}$$

GammaNet

Other options to train GammaNet:

- REINFORCE algorithm Policy Update

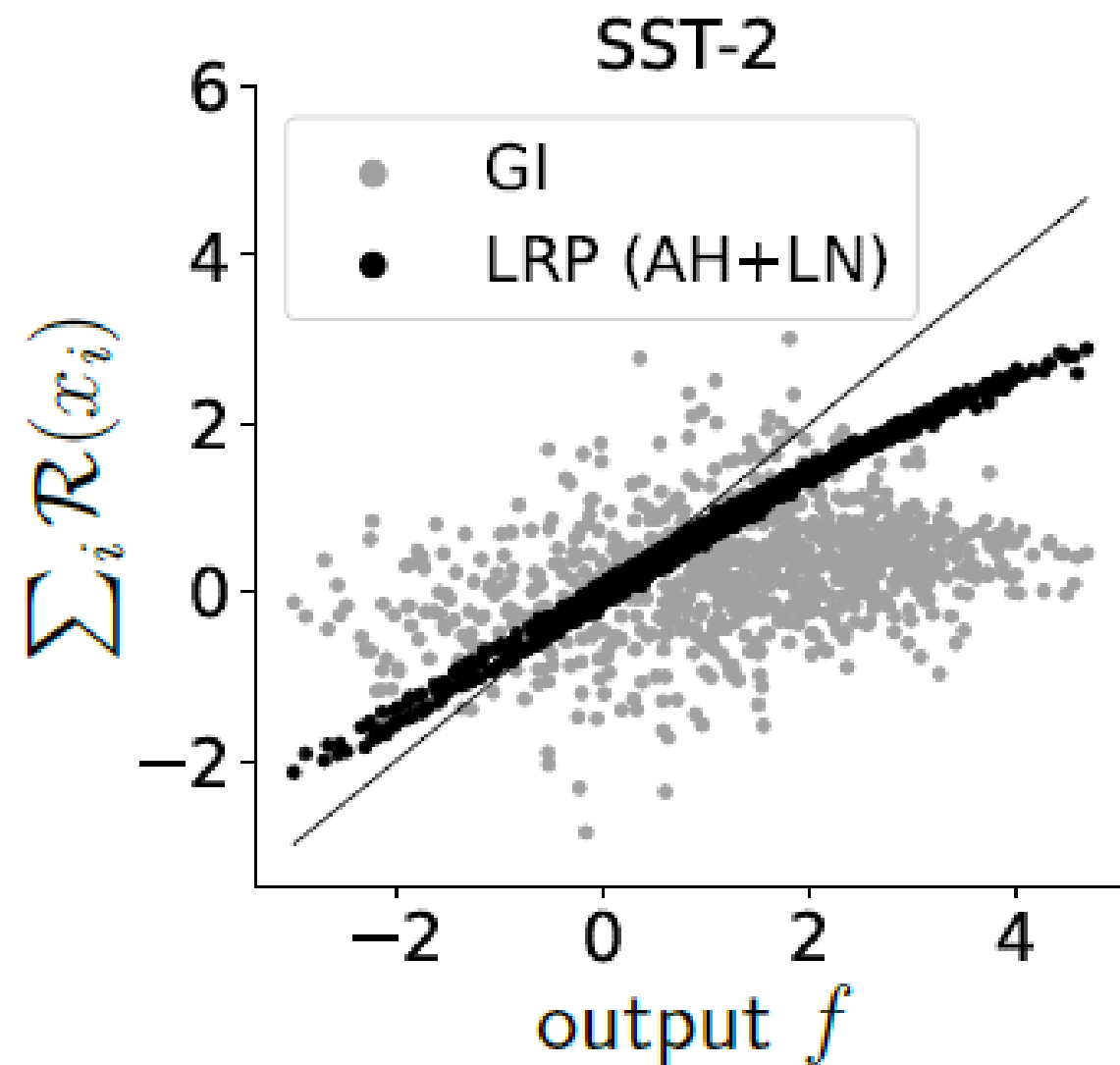
GammaNet

- We train this network on the SST train split.
- Training is highly unstable.
 - Gradient clipping to ensure $[0, 1]$ range
 - AUAC proxy
 - We want complete gradient propagation to estimate gammas, but only partial flow to allow for expressivity-completeness trade-off.

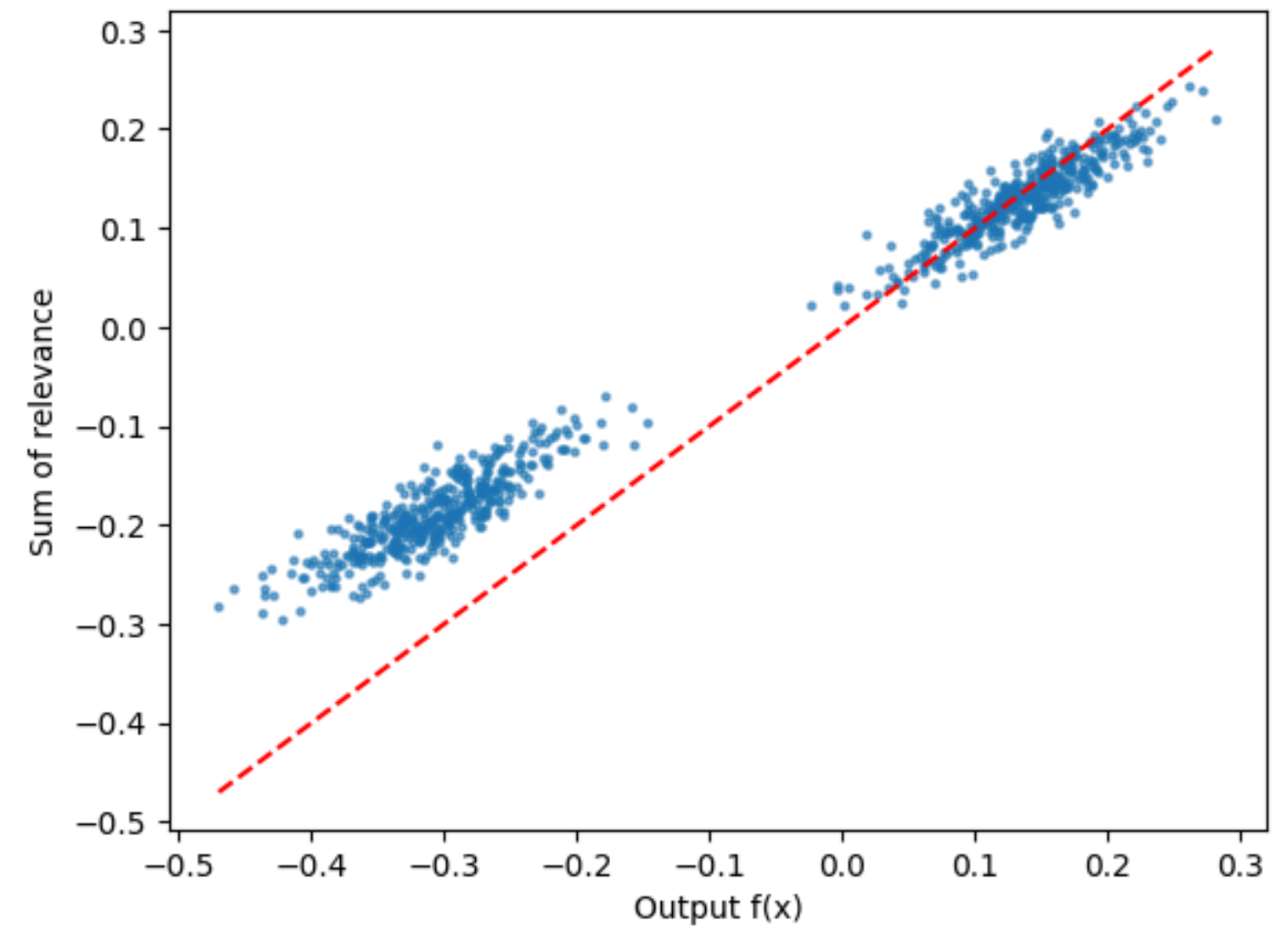
Results

Completeness:

From the paper

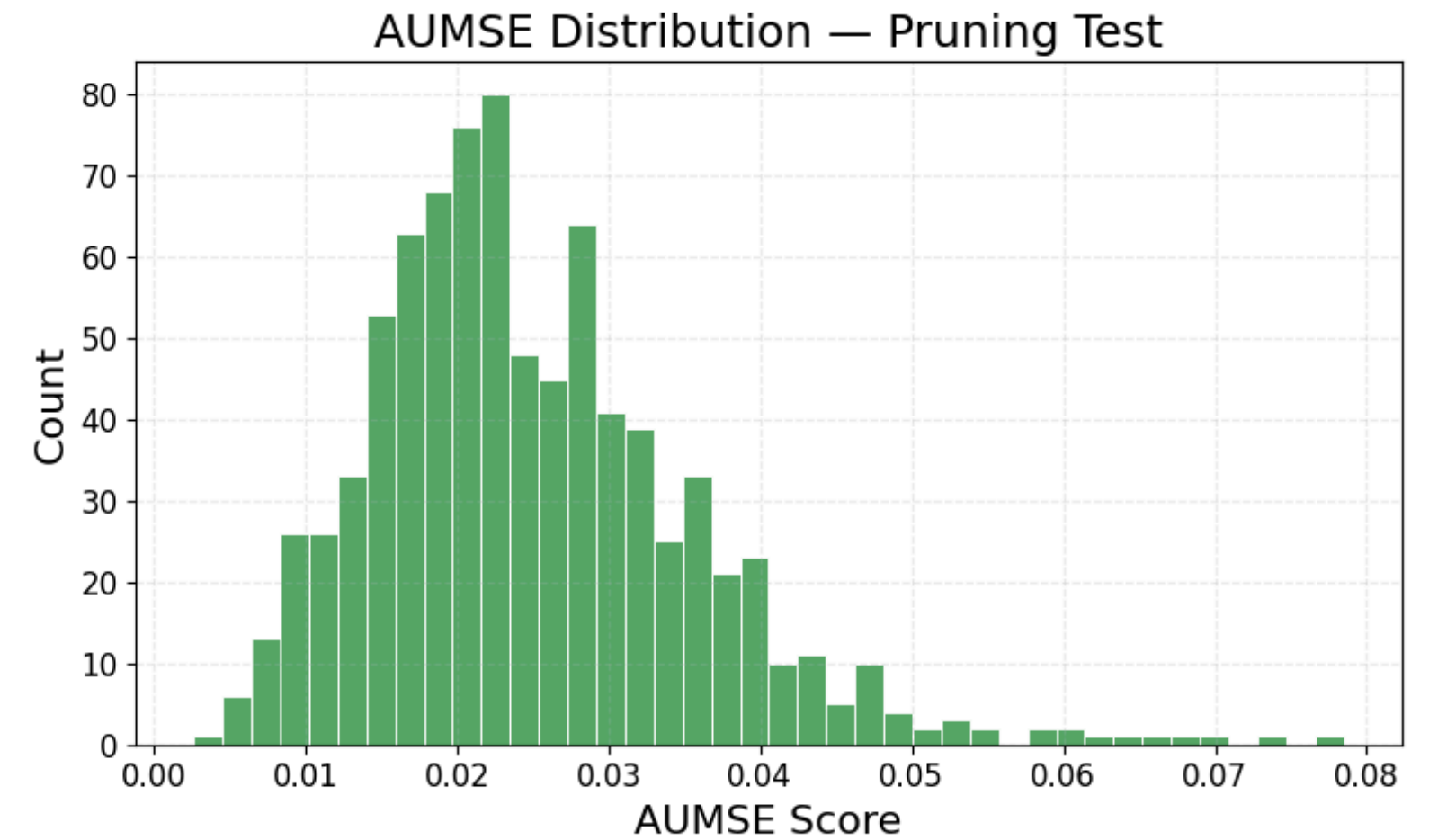
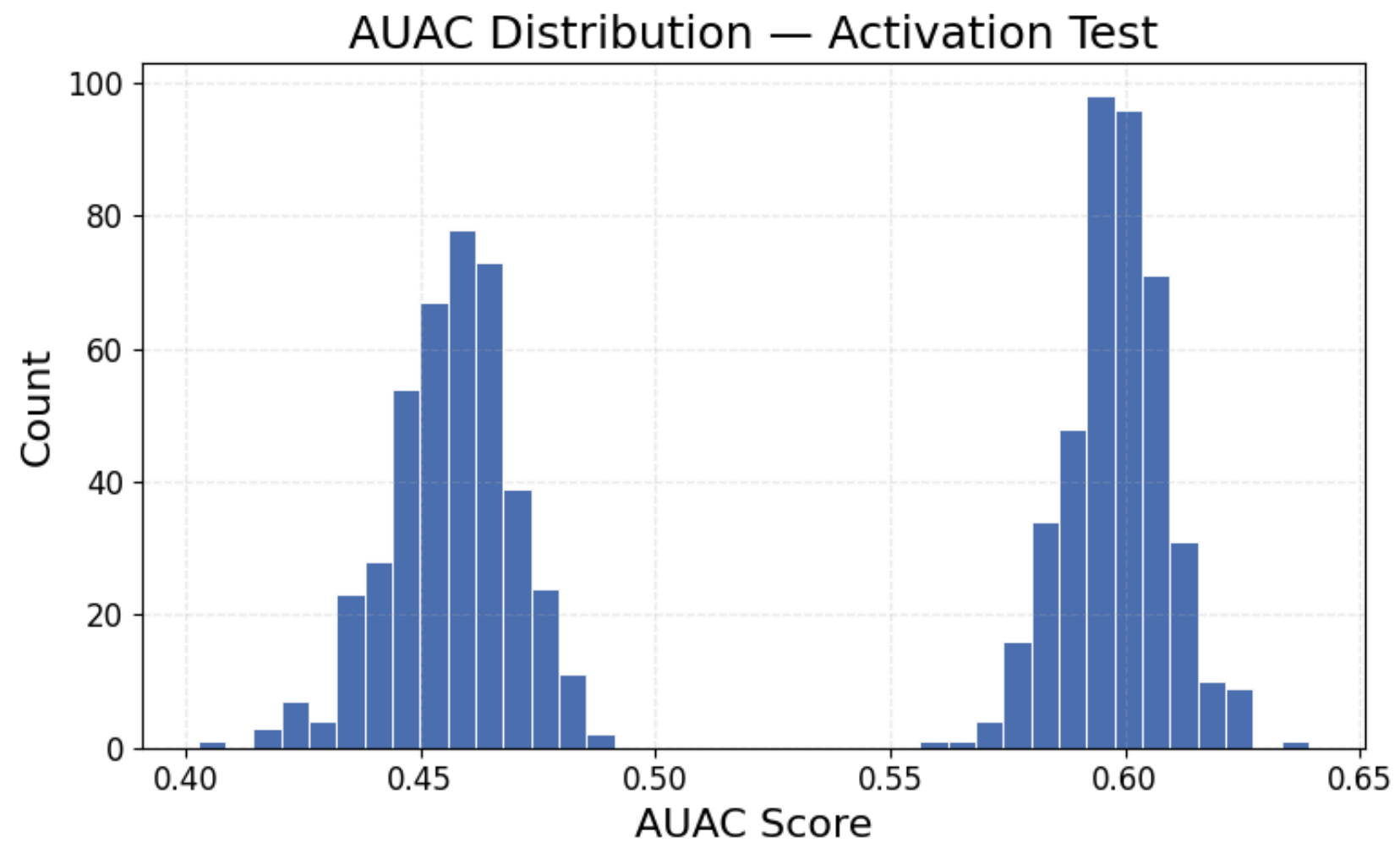


GammaNet



Results

Expressivity:



FUTURE WORK

1. Learn Per-Head Gammas Instead of Per-Layer
2. Learn Per-Token Gammas
3. Use a Fully Differentiable Faithfulness Loss

REFERENCES

Our work is heavily inspired from, and builds on top of:

[1] Ali, Ameen, et al. XAI for Transformers: Better Explanations through Conservative Propagation. n.d.

Source Code

All of our work has been open sourced and is available at:
https://github.com/nikhil-405/XAI_for_Transformers